

C++ Notes

Item 9 — Prefer alias declarations to typedefs

Adam HAMOUC

May 2026

1. Définition

`typedef` et `using` permettent tous les deux de donner un nom court à un type long. En C++ moderne, **toujours préférer `using`** — même résultat, mais compatible avec les templates et plus lisible.

```
// typedef : syntaxe C++98
typedef std::unique_ptr<std::unordered_map<std::string, std::string>> UPtrMapSS;

// using : syntaxe C++11 moderne, equivalente mais superieure
using UPtrMapSS = std::unique_ptr<std::unordered_map<std::string, std::string>>;

// En pratique dans un moteur
using PipelineHandle = Handle<PipelineTag>;
using TextureCache   = std::unordered_map<uint64_t, std::shared_ptr<Texture>>;
using DrawList       = std::vector<DrawCall>;
```

2. Avantage clé : compatibilité avec les templates

`typedef` ne peut pas être templatisé directement. `using` si — c'est un **alias template**.

```
// typedef : oblige a wrapper dans une struct, lourd
template<typename T>
struct MyAllocList {
    typedef std::list<T> type; // "type" n'est pas un alias template
};
MyAllocList<Widget>::type list; // verbose

// using : alias template direct, propre
template<typename T>
using MyAllocList = std::list<T>;
MyAllocList<Widget> list; // direct, aucun ::type necessaire
```

3. Le problème du typename avec typedef

Quand on accède à `typedef::type` dans un template, le compilateur ne sait pas si `::type` est un **type** ou un **membre de données** — cela dépend de T, qui est inconnu. Il faut donc prefixer par `typename` pour lever l'ambiguïté.

Avec un alias template (`using`), ce problème disparaît : le compilateur sait toujours que c'est un type.

```
// Avec typedef : typename obligatoire dans un template
template<typename T>
class Widget {
    typedef MyAllocList<T>::type list; // typename requis
};
```

```
// Avec using : aucun typename necessaire
template<typename T>
class Widget {
    MyAllocList<T> list; // propre, direct
};
```

Pourquoi le compilateur ne peut pas deviner ? Une spécialisation partielle pourrait redéfinir `::type` comme un membre de données pour un type particulier. Le compilateur doit donc exiger `typename` pour se protéger. Avec `using`, ce cas n'existe pas.

4. Application : `type_traits` et `alias_t`

Les `type_traits` de la STL utilisaient historiquement `typedef` et `::type`, imposant `typename` dans les templates. C++14 a introduit des alias `_t` basés sur `using` pour éviter ce problème.

```
// C++11 : typedef interne, typename obligatoire dans un template
typename std::remove_const<T>::type          // const T  => T
typename std::remove_reference<T>::type      // T&      => T
typename std::add_pointer<T>::type           // T        => T*

// C++14 : alias _t, typename inutile
std::remove_const_t<T>
std::remove_reference_t<T>
std::add_pointer_t<T>

// Exemple concret
template<typename T>
void upload(T& data) {
    using CleanT = std::remove_const_t<std::remove_reference_t<T>>;
    static_assert(std::is_trivially_copyable_v<CleanT>);
    memcpy(mappedPtr, &data, sizeof(CleanT));
}
```

5. Récapitulatif

- `typedef` et `using` sont équivalents pour les types simples — préférer `using` par convention moderne.
- Seul `using` peut être templatisé directement (alias template).
- Avec `typedef::type` dans un template → `typename` obligatoire. Avec `using` → rien.
- Préférer les alias `_t` de C++14 (`remove_const_t`, `remove_reference_t`...) aux formes `::type` de C++11.